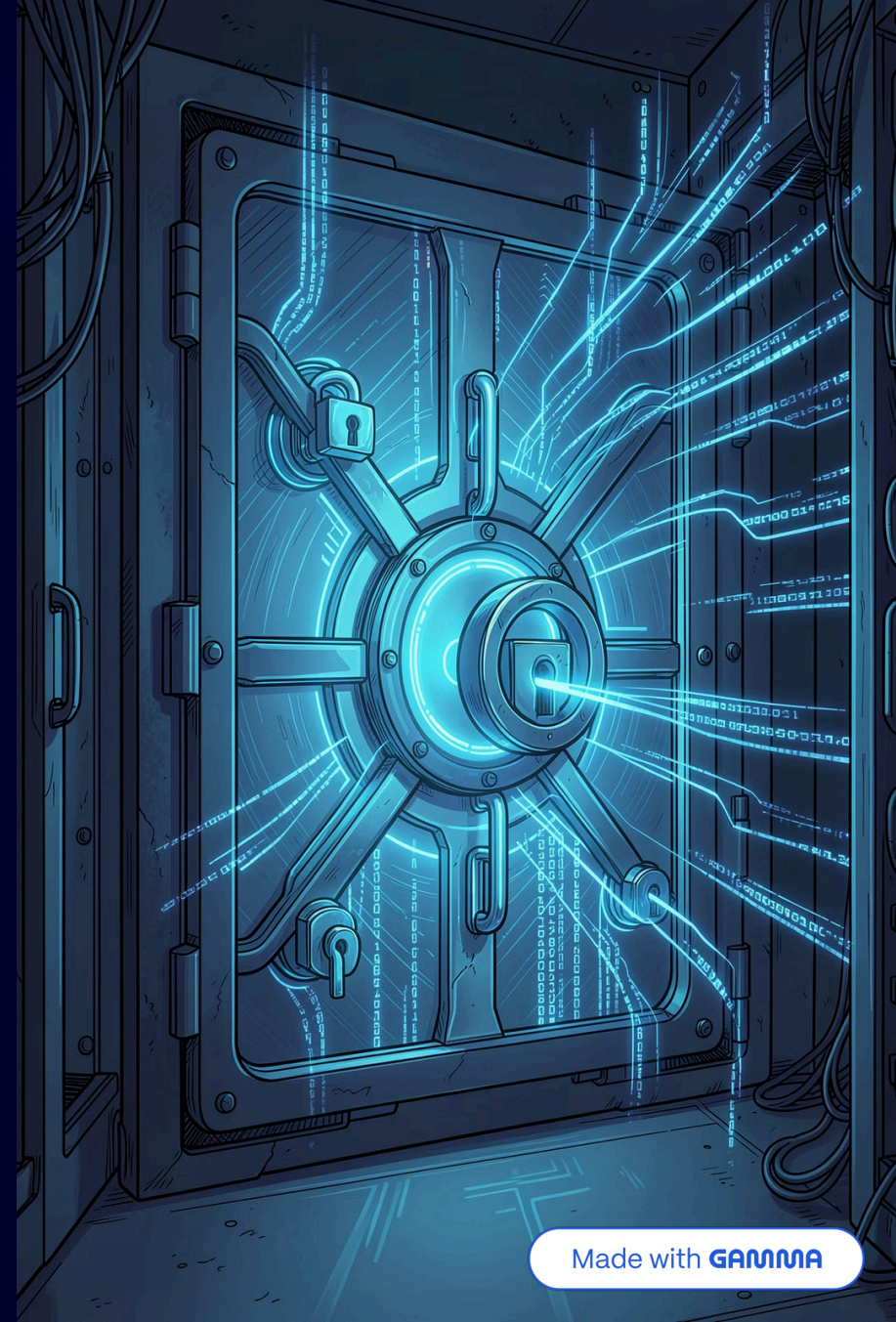


Administración en SQL Server

Usuarios, Permisos y Accesos

Diseña un acceso seguro como si fuera una caja fuerte: **mínimo privilegio** y trazabilidad total.



¿Qué es y para qué sirve?



La administración de usuarios y permisos en SQL Server controla la **autenticación** y la **autorización**, garantizando que cada identidad realice únicamente las acciones que necesita.

Evitar accesos indebidos

Restringir quién puede entrar y qué puede tocar.

Prevenir errores críticos

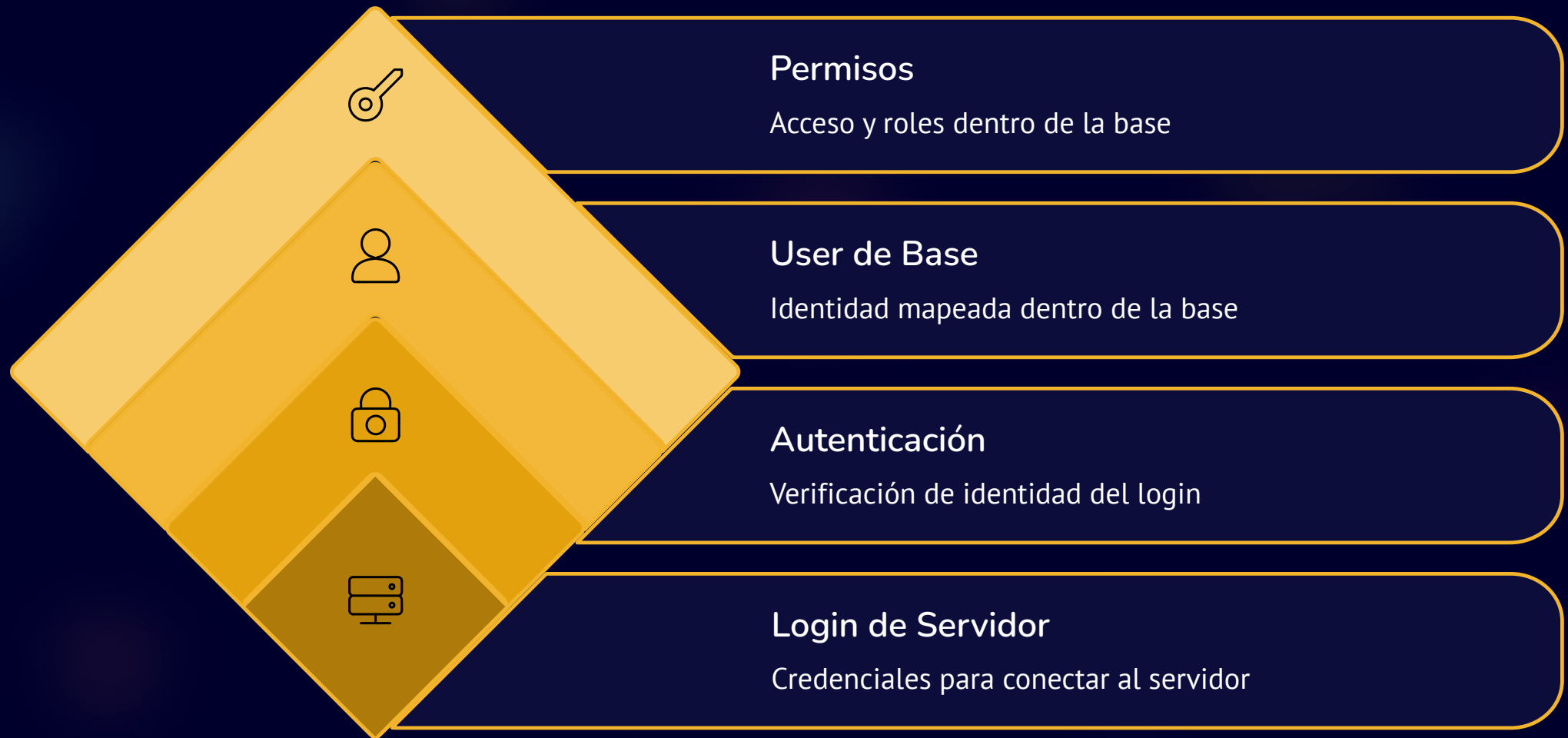
Evitar borrados accidentales o modificaciones no autorizadas.

Cumplir auditorías

Registrar trazabilidad de cada acción en el sistema.

¿Cómo funciona? La doble capa de seguridad

SQL Server protege los datos en **dos niveles independientes**. Comprender esta arquitectura es la base de todo control de acceso.



El **login** abre la puerta al servidor; el **user** define qué puedes hacer dentro de cada base de datos. Son entidades distintas que deben mapearse correctamente.

Entidades de seguridad: el mapa que manda

SQL Server organiza el acceso mediante **entidades de seguridad** clasificadas por ámbito. Los permisos se conceden a estas entidades, no directamente a personas.



Ámbito de servidor

Logins, roles de servidor, credenciales

Ámbito de base de datos

Users, roles de BD, esquemas

Ámbito de objeto

Tablas, procedimientos, vistas

Autenticación vs. Autorización: no es lo mismo



Autenticación

"¿Eres quién dices ser?"

Verifica la credencial al conectar al servidor. Sin login válido, no hay acceso.



Autorización

"¿Qué puedes hacer?"

Define las acciones permitidas sobre objetos dentro de la base de datos.



Auditoría

"¿Quién hizo qué?"

Registra y permite rastrear cada acción para control y cumplimiento normativo.



Estos tres pilares forman el **círculo completo de seguridad** en SQL Server. Omitir uno deja brechas críticas.

Permisos con intención: principio de mínimo privilegio

No entregues la llave de la caja fuerte a todos los empleados. Da solo la llave que necesitan para su trabajo.

→ Evalúa la necesidad real

¿Qué tareas realiza ese usuario o proceso? ¿Qué objetos necesita tocar?

→ Concede solo lo necesario

Evita `sysadmin` por comodidad. Cada permiso extra es un riesgo potencial.

→ Revisa periódicamente

Los permisos acumulados con el tiempo crean deuda de seguridad silenciosa.



Roles: la forma inteligente de gestionar el control



Asignar permisos individualmente a cada usuario es caótico y propenso a errores. Los **roles** agrupan permisos y se asignan a usuarios, simplificando la gestión y reduciendo inconsistencias.

db_owner

Control total sobre la base de datos.
Usar con extrema cautela.

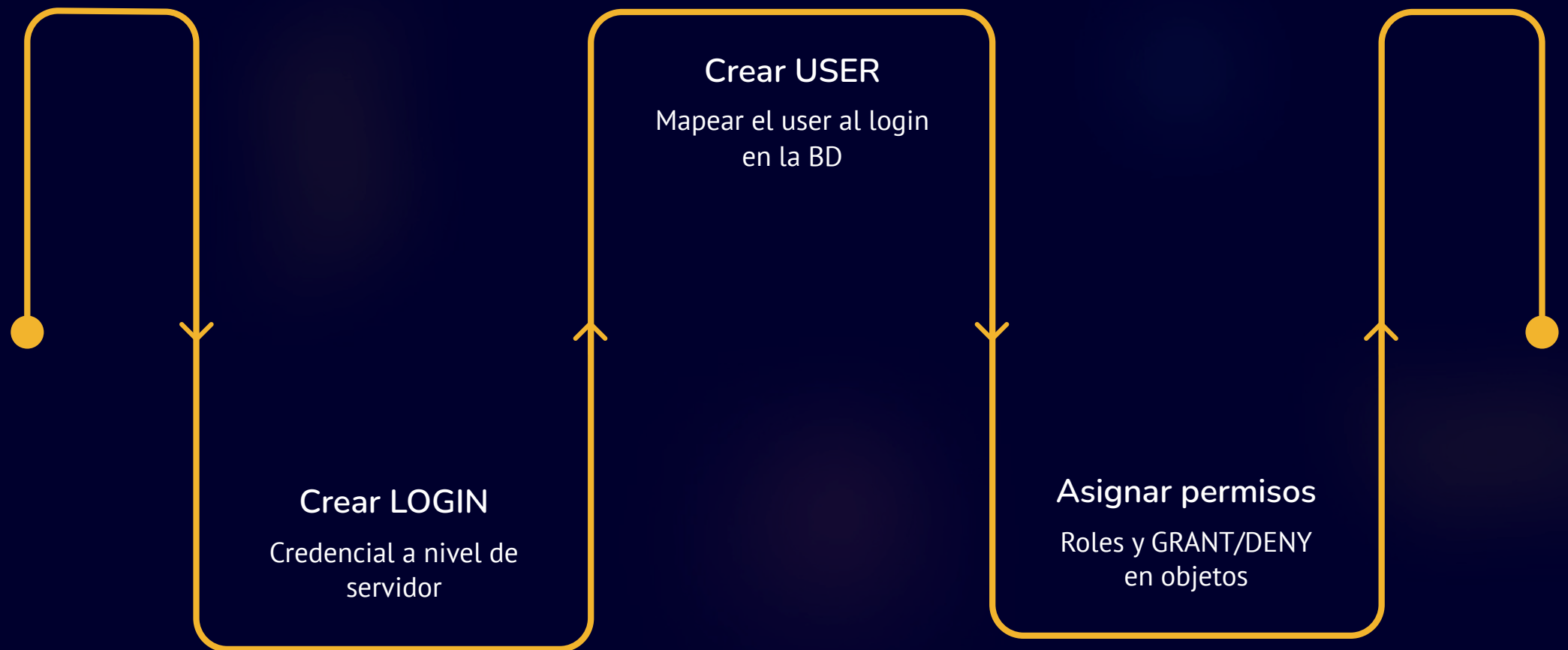
db_datareader

Lectura en todas las tablas y vistas.
Ideal para analistas.

db_datawriter

Escritura en tablas. Para procesos ETL o aplicaciones.

Aplicación paso a paso: Login → User → Permisos



Este flujo de tres pasos garantiza que cada identidad tenga acceso controlado y trazable desde el servidor hasta los objetos específicos.

Ejemplo T-SQL básico

```
-- 1. Crear login
CREATE LOGIN analista WITH PASSWORD = '***';

-- 2. Crear user en BD
USE Ventas;
CREATE USER analista FOR LOGIN analista;

-- 3. Asignar rol
ALTER ROLE db_datareader
ADD MEMBER analista;
```

⚠ **Recuerda:** Nunca concedas `sysadmin` "porque sí". Ese patrón es la causa más frecuente de auditorías fallidas y brechas de seguridad.

Simula una brecha y repárala

El escenario

Un nuevo analista necesita acceder a SQL Server para consultar y actualizar datos de ventas, pero **solo en tablas concretas**.

El requisito

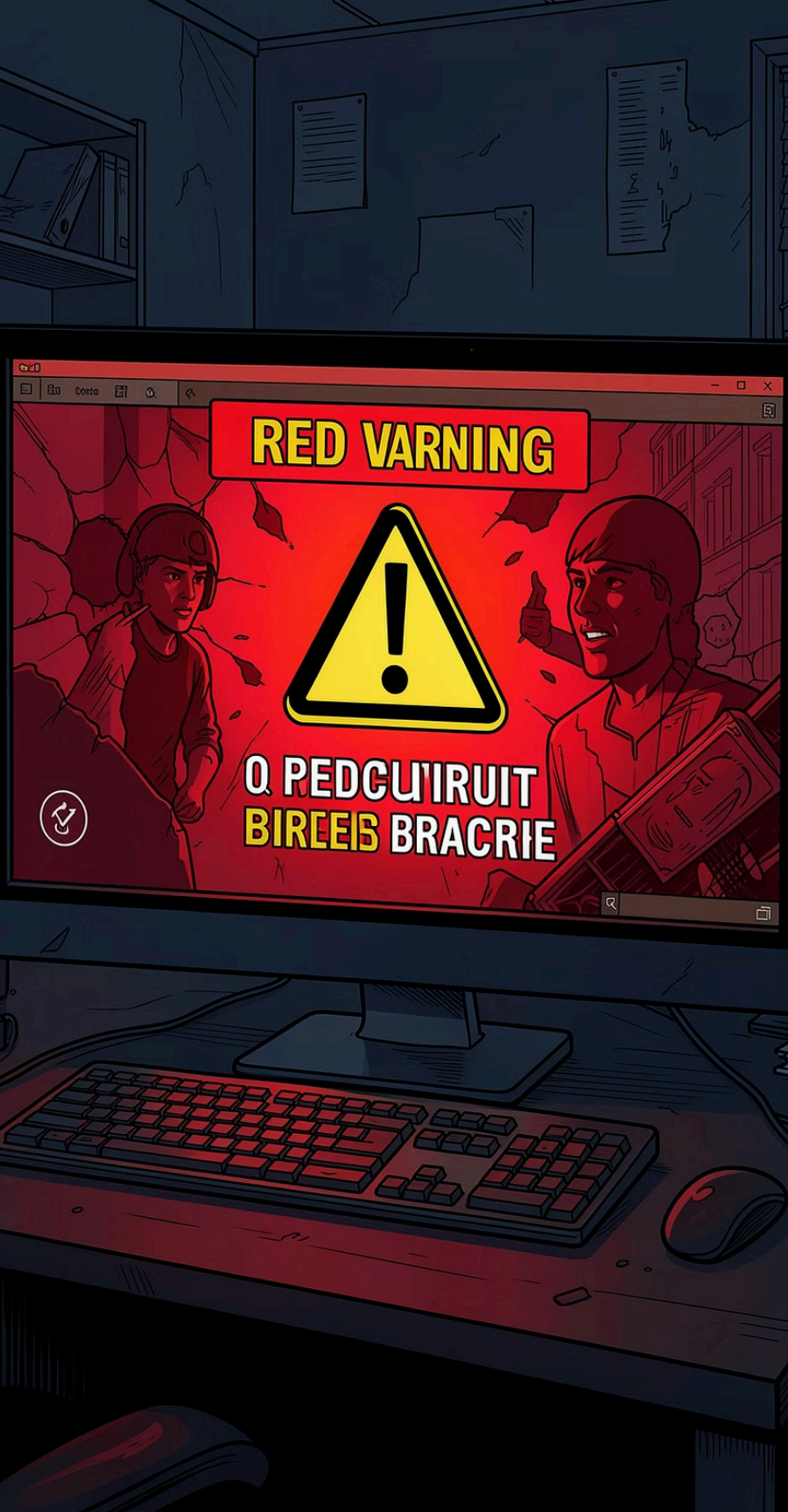
Crear **login** y **user** para "analista", permitiendo solo **SELECT** y **UPDATE** en las tablas **Ventas** y **Pedidos**.

La trampa a evitar

No conceder **db_owner** ni **sysadmin** por comodidad. Ese patrón causa **auditorías fallidas**.

Entrega esperada

Lista final de roles asignados + permisos explícitamente **permitidos** y **denegados** con justificación.



Conclusiones: el checklist que te salva en producción

01

Diseña por capas

Login (servidor) + User (base de datos). Nunca los confundas ni los saltes.

03

Usa roles, no asignaciones sueltas

Los roles predefinidos (`db_datareader`, `db_datawriter`) reducen errores y simplifican auditorías.

02

Aplica mínimo privilegio

Cada permiso debe tener una razón documentada. Si no se usa, se revoca.

04

Activa auditoría y buenas prácticas

Seguridad completa = autorización + trazabilidad + configuración hardened del servidor.



Regla de oro: Si no puedes justificar un permiso en una auditoría, no deberías haberlo concedido.